

Soluciones Tecnológicas para el Transporte Público Urbano

Samuel Rioja

CTO & Solution Architect
NEXION S.R.L.

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz





AGENDA

Lo que veremos en esta presentación

Recorrido completo por la arquitectura del sistema MiRuta Trujillo: los servicios que corren en producción, el pipeline de datos en tiempo real, la app móvil y las operaciones del día a día.

01

p. 3 – 8

Conceptos básicos

- Docker & docker-compose
- SSH y autenticación por llave
- Reverse proxy
- Git submodules
- SSL / Let's Encrypt

02

p. 9 – 14

Datos & formatos

- GTFS estático
- GTFS-RT (tiempo real)
- `trip_id` — el problema central
- OSM — OpenStreetMap
- Vector tiles & MapLibre
- GraphQL

03

p. 15 – 21

Arquitectura & OTP

- Diagrama del sistema
- Stack de servicios
- Demo: health check global
- OTP overview, driver app
- Configuración & GraphQL

04

p. 22 – 33

Pipeline + Servicios

- GTFS-RT proxy, enrich, simulator
- Photon (geocoding)
- TileServer GL (mapas)
- YARP / proxy + analytics
- web-landing, datos de input

05

p. 34 – 46

App móvil

- Stack: Flutter + trufi-core
- Build & release
- Admin: 9 tareas comunes (versión, URLs, branding, idioma, etc.)

06

p. 47 – 75

Operaciones

- Troubleshooting (10 problemas comunes)
- Cheat sheet (deploy, debug, SQL)
- Gestión de accesos & SSH
- Backup & disaster recovery (8 categorías)
- Cierre & contacto

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit





Docker — cajas aisladas para programas

Una "caja virtual" que contiene un programa con todo lo que necesita: sistema operativo mínimo, librerías y configs. Cada servicio del stack corre en su propia caja.

¿POR QUÉ IMPORTA?

En vez de instalar OTP, Photon, PostgreSQL, etc. directamente en el server, cada uno corre en su propia caja Docker.

- Si uno falla, no rompe a los demás
- Versiones independientes — actualizas OTP sin tocar Photon
- Reproducible: lo mismo en dev, staging y prod
- Fácil rollback: `docker compose down` + redeploy

COMANDOS ESENCIALES

```
# Ver cajas corriendo
docker ps

# Reiniciar una caja
docker restart <nombre>

# Ver lo que imprime
docker logs -f <nombre>

# Logs últimos 100
docker logs --tail 100 <nombre>

# Entrar a la caja con shell
docker exec -it <nombre> bash
```



docker-compose — orquestar varias cajas juntas

Un archivo `docker-compose.yml` describe varios containers Docker como un grupo: qué imagen, qué puertos, qué volúmenes, qué red comparten. Permite levantarlas todas con un solo comando.

ESTRUCTURA DE CADA SERVICIO

En el server hay un folder por servicio, cada uno con su propio compose:

```
/root/
├── otp/           docker-compose.yml
├── server/        docker-compose.yml
├── photon/        docker-compose.yml
├── tileserver-gl/ docker-compose.yml
├── gtfs-rt-proxy/ docker-compose.yml
└── web-landing/   docker-compose.yml
```

Todos comparten la red `trufi-server` para hablarse entre sí.

LOS 3 COMANDOS QUE USAS 99% DEL TIEMPO

```
cd /root/<servicio>

# Levantar (build + start en background)
docker compose up -d --build

# Reiniciar (sin rebuild)
docker compose restart

# Bajar
docker compose down

# Ver el estado
docker compose ps

# Logs en vivo
docker compose logs -f
```




SSH — Secure Shell

Protocolo para ejecutar comandos en una computadora remota desde tu terminal local. Como tener una consola que está físicamente dentro del servidor — pero encriptada y autenticada por llave.

CÓMO FUNCIONA LA AUTENTICACIÓN

Generas un **par de llaves**: una pública y una privada. La pública la subes al server (al archivo `~/.ssh/authorized_keys` del usuario remoto). La privada se queda en tu máquina y nunca sale.

Cuando conectas, SSH usa criptografía para demostrar que tienes la llave privada que coincide con esa pública — sin transmitirla.

- Sin password — más seguro y más rápido
- La llave privada está protegida por contraseña local (passphrase)
- Puedes agregar/quitar accesos editando `authorized_keys`

SETUP Y USO

```
# 1. Generar par de llaves (en tu máquina)
ssh-keygen -t ed25519 -C "tu-email@ejemplo.com"

# 2. Copiar la pública al portapapeles
cat ~/.ssh/id_ed25519.pub | pbcopy # macOS
cat ~/.ssh/id_ed25519.pub | xclip  # Linux

# 3. (un admin ya autorizado) la agrega:
echo "ssh-ed25519 AAAA... tu-email" \
  >> /root/.ssh/authorized_keys

# 4. Conectarte
ssh root@trujillo.trufi.dev
```



Reverse proxy — el guardián de la entrada

Un programa que recibe TODAS las peticiones públicas (puerto 80/443) y las redirige al servicio interno correcto. En este stack lo cumple **YARP** (.NET 10).

POR QUÉ IMPORTA

Solo el reverse proxy escucha públicamente. OTP, Photon, TileServer, etc. están *escondidos* detrás — no son accesibles directamente desde internet.

Si alguien hace `GET trujillo.trufi.dev/otp/...`, YARP lo manda internamente a `http://otp:8080/...`.
Beneficios:

- Un solo certificado SSL/TLS para todo el stack
- Logs centralizados de todos los requests
- No exponer servicios sensibles a internet
- Fácil sumar/quitar backends sin tocar DNS
- Rate limiting, auth, headers — en un solo lugar

ROUTING EN ESTE STACK

```
Cliente
| HTTPS (puerto 443)
v
trujillo.trufi.dev
|
v
┌─────────── YARP (reverse proxy) ───────────┐
│ - SSL termination                         │
│ - request logging                         │
└───────────┘
|
├── /otp/* → http://otp:8080
├── /photon/* → http://photon:2322
├── /tiles/* → http://tileserver-gl
└── / → http://web-landing
```



Git submodules — incluir otros repos como subcarpetas

Un repo git puede "incluir" otros repos como subcarpetas, fijando un commit específico de cada uno. Permite versionar qué versión del código externo usas sin duplicarlo en tu propio repo.

POR QUÉ LOS USAMOS

OTP, Photon, TileServer GL y YARP/server son repos de trufi-association (mantenidos por la comunidad Trufi). No son nuestros, pero los necesitamos en producción.

Los traemos como submódulos para:

- Versionar exactamente qué commit corre en producción
- Poder upgradar (o revertir) controladamente
- No duplicar miles de líneas de código en este repo
- Mantener attribution clara de upstream

COMANDOS COMUNES

```
# Clonar este repo CON submódulos
git clone --recurse-submodules <url>

# Si ya cloneaste sin submódulos
git submodule update --init --recursive

# Actualizar a últimos commits remotos
git submodule update --remote --merge

# Ver estado de cada submódulo
git submodule status

# Cambiar al main de un submódulo específico
cd server/otp
git checkout main
git pull
```



SSL / TLS y Let's Encrypt

El candadito verde  en el browser. Necesario para que la conexión cliente ↔ servidor sea encriptada (HTTPS) y para evitar ataques man-in-the-middle. Sin esto, la app móvil no podría hablar con OTP de manera segura.

CÓMO SE OBTIENE UN CERTIFICADO

Let's Encrypt es una CA (Certificate Authority) gratis y automatizada. En lugar de pagar a Comodo o DigiCert, te da certificados válidos de 90 días, renovables sin límite.

El proceso (ACME):

- Tu server prueba que controla el dominio (HTTP-01 o DNS-01)
- Let's Encrypt emite el certificado
- YARP lo guarda y lo presenta en cada request HTTPS
- 30 días antes de expirar, lo renueva automáticamente

LETTUCEENCRIPT – LA INTEGRACIÓN CON YARP

LettuceEncrypt es la librería .NET que YARP usa para gestionar el ciclo de vida de certificados Let's Encrypt. Cero config manual.

```
// Program.cs (resumido)
builder.Services
    .AddLettuceEncrypt(opts => {
        opts.AcceptTermsOfService = true;
        opts.DomainNames = new[] {
            "trujillo.trufi.dev"
        };
        opts.EmailAddress = "admin@...";
    })
    .PersistDataToDirectory(
        new DirectoryInfo("/data/lettuce"),
        pfxPassword: null);
```

Los certificados persisten en `/root/server/data/lettuce-encrypt/` (backupear si reinstalas).



GTFS · DATOS · 1 / 6

GTFS — General Transit Feed Specification

Estándar abierto de Google (2006) para describir **transporte público estático**: qué líneas existen, dónde están los paraderos, qué horarios siguen.

Formato: un `.zip` con archivos `.txt` en CSV.

TABLAS PRINCIPALES

- `routes.txt` — qué líneas existen (Línea 19, etc.)
- `stops.txt` — paraderos (lat/lon, nombre)
- `trips.txt` — un viaje específico de una línea
- `stop_times.txt` — horario de cada parada de cada trip
- `frequencies.txt` — "cada 5 min sale un bus" (frecuencias en lugar de horarios fijos)
- `calendar.txt` — qué días opera (lun-vie / fines de semana)
- `shapes.txt` — geometría de las rutas (puntos lat/lon)

EL GTFS DE TRUJILLO

Vive en `server/data/trujillo.gtfs.zip` (18 MB). Lo consumen **OTP** al levantarse para construir el graph de routing.

Particularidad: usa `frequencies.txt` con `exact_times=1`. Eso significa que en lugar de listar cada bus de las 6:00, 6:05, 6:10..., el GTFS define una "plantilla" de trip y dice "se repite cada 5 min entre 5:30 y 22:00".

Esa decisión de modelado tiene una consecuencia importante en GTFS-RT — la vemos en la slide del `gtfs-rt-proxy`.



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale Zusammenarbeit





GTFS · DATOS · 2 / 6

GTFS-RT — GTFS Realtime

Extensión del estándar GTFS para datos **en vivo**: dónde están los buses en este momento, retrasos en tiempo real, alertas de servicio. Formato: **protobuf** (binario, no texto), llamado típicamente `.proto`.

3 TIPOS DE MENSAJES

- **Vehicle positions** — bus X está en lat/lon, llamando route_id, direction_id
- **Trip updates** — bus X tendrá 5 min de retraso al llegar a la próxima parada
- **Service alerts** — "ruta 19 cancelada hoy por desfile"

En Trujillo solo tenemos **vehicle positions** — los choferes reportan su ubicación cada pocos segundos.

EL FEED DE TRUJILLO

El driver app (otra app, no la de pasajeros) publica el feed a S3:

```
https://trujillo-admin-storage.s3...  
/driver/gtfsrt/pe-trujillo.proto
```

OTP **no consume directo** ese feed. Pasa primero por el `gtfs-rt-proxy`, que lo enriquece (le agrega los `trip_id` que faltan) antes de entregárselo a OTP cada 30s.

Por qué hace falta el proxy → ver slide siguiente.



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





GTFS · DATOS · 3 / 6

trip_id — el problema central que resuelve este sistema

Cada viaje específico de un bus tiene un `trip_id` único en el GTFS estático. Pero el driver app de Trujillo reporta solo `route_id + direction_id + start_time` — no el `trip_id`. OTP necesita el `trip_id` para predecir ETAs.

POR QUÉ OTP LO NECESITA

Cuando OTP recibe una posición de bus, intenta matchear ese bus con un trip específico del GTFS. Si encuentra el trip, puede:

- Calcular qué tan adelantado/retrasado va respecto al horario
- Predecir cuándo llegará a las paradas siguientes
- Ofrecer ETAs precisos a la app móvil

Sin el `trip_id`, OTP solo sabe "hay un bus por ahí en la línea 19" — no "el bus que sale a las 14:30 va con 5 min de retraso".

EL TWIST DE LAS FRECUENCIES

Como el GTFS de Trujillo usa `frecuencias.txt` con `exact_times=1`, cada "trip plantilla" representa decenas de buses reales (uno cada 5 min).

El proxy resuelve esto **expandiendo** los trips:

```
# Antes (GTFS original):
trip_id = 19962292    ← plantilla cada 5 min

# Después (expandido):
19962292000000    ← bus de las 5:30
19962292000001    ← bus de las 5:35
19962292000002    ← bus de las 5:40
...
19962292000069    ← bus de las 11:15
```

El proxy infiere a qué slot pertenece cada vehículo cruzando `route_id + direction_id + start_time` con la tabla de frecuencias.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz





GTFS · DATOS · 4 / 6

OSM — OpenStreetMap

El "Wikipedia de los mapas". Datos cartográficos libres y editables colaborativamente, con licencia ODbL. Cada calle, edificio, parque, río del mundo. La fuente única de geometría que usa todo el stack.

DE DÓNDE VIENE EN ESTE PROYECTO

El extract de Trujillo se genera con **trufi-pbf-extractor**, herramienta de Trufi Association que automatiza el recorte de OSM por ciudad.

CÓMO FUNCIONA LA HERRAMIENTA

```
# 1. Configurar .env
CITY=trujillo
BBOX=-79.10,-8.20,-78.95,-8.05
GEOFABRIK_URL=https://download.geofabrik.de/...

# 2. Levantar el container
docker compose up

# Adentro hace:
# - download del .pbf país desde Geofabrik
# - osmium extract con el bbox configurado
# - escupe trujillo.osm.pbf en data/
```

El extract final vive en `server/data/trujillo.osm.pbf` (2.3 MB) — formato `.osm.pbf` (Protocol Buffer comprimido).

QUIÉN LO CONSUME EN EL STACK

- **OTP** — construye el grafo de calles caminables/ciclables (necesario para multi-modal: caminata + bus + caminata)
- **TileServer GL** — los `.mbtiles` que sirve son tiles vectoriales generados desde el mismo OSM
- **Photon** — indexa los nombres de calles, lugares y POIs para geocoding

Una sola fuente de verdad geográfica: cuando regeneramos el OSM con `trufi-pbf-extractor`, hay que regenerar los derivados (graph de OTP, mbtiles, índice de Photon) para que el sistema quede consistente.



trufi-association/trufi-pbf-extractor

Docker + osmium-tool · genera el .osm.pbf de cada ciudad



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





Vector tiles + .mbtiles

Los mapas modernos no se sirven como imágenes pre-pintadas (raster), sino como pequeños fragmentos de geometría que el cliente *renderiza* con código. Más livianos, infinitamente zoomables, retematizables al vuelo.

VECTOR TILES VS RASTER

Raster (clásico): el server pinta una imagen 256×256 PNG y la manda al cliente. Cada zoom requiere un set distinto. Pesado y rígido.

Vector tiles: el server manda solo geometría (líneas, polígonos, puntos) en un formato binario eficiente (Mapbox Vector Tile). El cliente las dibuja con WebGL/MapLibre.

- Texto siempre nítido (no pixela al hacer zoom)
- Cambiar tema (oscuro / claro / satélite) sin descargar tiles nuevos
- ~10× más liviano que raster equivalente
- Renderizado del lado del cliente — server más liviano

El formato `.mbtiles` es una base SQLite con TODOS los tiles de una región indexados por (z, x, y). Trujillo entero cabe en **4.7 MB**.

PIPELINE EN ESTE PROYECTO

Generación y serving cubiertos por dos repos de Trufi Association, que arriba envuelven herramientas oficiales:

- **trufi-mbtiles-generator** — wrapper Docker Compose sobre Planetiler con perfil OpenMapTiles (imagen oficial [openmaptiles/planetiler-openmaptiles](#))
- **trufi-server-tileservier-gl** — empaqueta TileServer GL como container del stack

```
# Flujo end-to-end:
trujillo.osm.pbf
  ↓ trufi-mbtiles-generator
  ↓ (Planetiler + OpenMapTiles)
trujillo.mbtiles (4.7 MB)
  ↓ trufi-server-tileservier-gl
  ↓ (TileServer GL)
GET /tiles/{z}/{x}/{y}.pbf → MapLibre
```

MapLibre GL es la librería que la app móvil usa para renderizar — fork open-source de Mapbox GL (que se cerró en 2020).



openmaptiles/openmaptiles

Schema oficial · define las capas y atributos del `.mbtiles`



trufi-association/trufi-mbtiles-generator

Wrapper Docker Compose sobre Planetiler + OpenMapTiles





GraphQL — cómo la app habla con OTP

Lenguaje de consulta para APIs (Facebook, 2015). En vez de muchos endpoints REST, hay un **solo endpoint** que recibe queries donde el cliente pide exactamente los campos que necesita — ni más, ni menos.

POR QUÉ GRAPHQL Y NO REST

- El cliente decide qué campos quiere — la app móvil ahorra ancho de banda pidiendo solo lo necesario
- Una versión nueva de la app puede pedir más campos sin romper la API ni necesitar versionado de endpoints
- OTP no necesita `/v1/`, `/v2/`, etc.
- Schema fuertemente tipado y auto-documentado — IDE autocomplete
- Una sola request puede traer datos de varias entidades relacionadas (no más N+1)

EJEMPLO DE QUERY A OTP

```
POST /otp/routers/default/index/graphql
Content-Type: application/json

{
  "query": "{
    plan(
      from: { lat: -8.11, lon: -79.03 },
      to: { lat: -8.10, lon: -79.05 },
      transportModes: [
        { mode: TRANSIT },
        { mode: WALK }
      ],
      numItineraries: 3
    ) {
      itineraries {
        duration
        walkDistance
        legs {
          mode
          startTime
          from { name }
          to { name }
          route { shortName longName }
        }
      }
    }
  }"
```


ARQUITECTURA DEL SISTEMA

Cómo se conectan las piezas

Todo corre en un único servidor con Docker. YARP enruta el tráfico HTTPS hacia los servicios internos. OTP es el cerebro; el `gtfs-rt-proxy` lo alimenta con datos en tiempo real ya enriquecidos.

CLIENTES



App móvil

Flutter · MapLibre · trufi-core



Landing web

HTML + CSS + JS · i18n ES/EN

----- HTTPS · puerto 443 -----

EDGE · TRUJILLO.TRUFI.DEV



YARP — reverse proxy

.NET 10 · routing · request logging · :80 :443

HEALTHY



Let's Encrypt

SSL · LettuceEncrypt

----- routing por path -----

CORE SERVICES



OTP

OpenTripPlanner · :8080



Photon

Geocoding · :2322



TileServer GL

Vector tiles · interno



web-landing

nginx · estático

----- OTP polls · cada 30s -----

DATA PIPELINE · TIEMPO REAL



gtfs-rt-proxy

Python + Flask · enriquece `trip_id` · :8001

CRÍTICO



gtfs-rt-simulator

Fallback dev / demos · :9000



PostgreSQL

Analytics — request logs

----- poll · cache 5s -----

EXTERNAL · DRIVER APP



Amazon S3

`trujillo-admin-storage/driver/gtfsrt/pe-trujillo.proto` · vehicle positions

6 SERVICIOS + 1 SIMULADOR

Lo que corre en el servidor

Cada uno en su container Docker, con su healthcheck, su repo y su rol específico en el pipeline. Vamos a verlos uno por uno.



OpenTripPlanner

:8080

El cerebro del sistema. Java 21. Ruteo multimodal con tiempo real. Consume GTFS estático + GTFS-RT enriquecido del proxy.

healthy v 2.8.1 submódulo



gtfs-rt-proxy

:8001

Servicio crítico custom. Python + Flask. Inyecta el `trip_id` que falta en el feed GTFS-RT del driver app antes de entregárselo a OTP.

healthy crítico en este repo



Photon

:2322

Geocoding ("dónde está..."). Java. Indexa OSM para resolver direcciones a lat/lon. Índice de Perú completo.

healthy v 0.4.2 submódulo



TileServer GL

interno

Sirve los vector tiles que renderiza la app móvil con MapLibre. Lee de `trujillo.mbtiles`. 6 estilos disponibles.

healthy node submódulo



YARP / trufi-server

:80 :443

Reverse proxy edge. .NET 10. TLS automático con Let's Encrypt, routing por path, request logging a PostgreSQL para analytics.

healthy .NET 10 submódulo



web-landing

público

Página estática que se sirve en `trujillo.trufi.dev/`. nginx + HTML/CSS/JS vanilla, i18n ES/EN.

healthy nginx:alpine en este repo



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz



► DEMO EN VIVO • ~15 MIN

Conectémonos al servidor y veamos el estado real

Antes de meternos en cada servicio uno por uno, abramos una terminal y mostremos cómo se ve la cosa "ahora mismo": qué corre, cómo se siente la máquina, qué dicen los logs.

- 01 Conectar: `ssh root@trujillo.trufi.dev`
- 02 Listar containers: `docker ps --format "table {{.Names}}\t{{.Status}}"`
- 03 Recursos del host: `df -h` · `free -h` · `uptime`
- 04 Logs en vivo del proxy: `docker logs -f gtfs-rt-proxy` (~30s)
- 05 Health endpoint: `curl http://127.0.0.1:8001/healthz | jq`
- 06 Identificar los *healthchecks* "falsos negativos" (server-server-1, tileserver)
- 07 Layout en disco: `ls /root/` — un folder por servicio



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





SERVICIO · OTP :8080

OpenTripPlanner 2.8.1 — el cerebro del ruteo

Construye un grafo de la ciudad combinando OSM + GTFS, y resuelve viajes multimodales (transporte + caminata + bici). Java 21 sobre `eclipse-temurin:21-jre-jammy`. Memoria por defecto 2 GB.

INPUTS CANÓNICOS

- `trujillo.gtfs.zip` — GTFS estático (18 MB)
- `trujillo.osm.pbf` — OSM extract (2.3 MB)
- GTFS-RT vía `gtfs-rt-proxy` (poll 30s)

OUTPUT REGENERABLE

- `graph.obj` — grafo serializado (~20 MB)
- Se construye en primer arranque (~2-3 min)
- Subsecuentes arranques: `--load` («segundos)

API GRAPHQL

```
POST /otp/routers/default/index/graphql
POST /otp/gtfs/v1 (alias OTP 2.x)

# Ejemplo
{ "query": "{ routes { gtfsId shortName longName } }" }
```

HEALTHCHECK

```
GET /otp/

# Configurado en compose
wget --spider http://localhost:8080/otp/
interval: 30s · start_period: 120s
retries: 3
```

VERSIONES SOPORTADAS

- **2.8.1** ← actual (recomendada)
- 2.7.0, 2.4.0, 2.2.0, 2.0.0
- 1.5.0 (legacy, no GraphQL)

Cambiar versión: `otpversion=2.7.0 docker compose up -d --build`

NETWORKING

Red Docker `trufi-server`. OTP es alcanzable desde otros containers como `http://otp:8080`.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale Zusammenarbeit





EL ORIGEN DE LOS DATOS EN VIVO

La driver app — la app que usan los conductores

Toda la información en tiempo real (¿dónde está cada bus ahora mismo?) viene de una **app móvil distinta**: la que llevan los conductores. Es el "upstream" de nuestro pipeline. No la mantenemos nosotros — pero entender qué publica es esencial para diagnosticar el sistema.

📱 QUÉ HACE

- Cada conductor abre la app antes de iniciar su recorrido
- Selecciona la línea + dirección (ej. "19" → "Centro")
- Reporta posición GPS cada pocos segundos
- Cierra la app al terminar el recorrido

☁️ CÓMO PUBLICA

- API que persiste en S3
- Bucket: `trujillo-admin-storage`
- Archivo: `driver/gtfsrt/pe-trujillo.proto`
- Formato: GTFS-Realtime protobuf binario
- Mensaje: *vehicle positions* con `route_id` + `direction_id` + `start_time` + lat/lon

⚠️ LO QUE NO TRAE

- **No** trae `trip_id` (el viaje específico)
- **No** trae predicciones de tiempo
- **No** trae alertas de servicio

Por eso necesitamos el `gtfs-rt-proxy` : para enriquecer el feed antes de que OTP lo consuma.

¿QUIÉN LA MANTIENE?

La driver app es de un equipo distinto (típicamente la autoridad de transporte o un contratista). Si los datos en tiempo real fallan en la app de usuario, el primer chequeo es `/healthz` del proxy: si `entities=0` , el problema está aguas arriba — en la driver app o en S3.

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





OTP · CONFIGURACIÓN

Los 2 archivos JSON que definen el comportamiento

OTP lee dos configs al arrancar. Pequeñas, pero deciden cómo se comporta el ruteo: penalidades de caminata, transferencias máximas, velocidades, timeout de la API.

OTP-CONFIG.JSON

Activa **features** opcionales del motor. Hoy solo tenemos uno: el endpoint GraphQL (que la app móvil consume).

· `GtfsGraphQLApi` · ☒ activo

Sin esta flag, la API legacy `/plan` sigue funcionando — pero la app móvil deja de andar.

ROUTER-CONFIG.JSON

Define **cómo** OTP planifica rutas. Las defaults actuales están afinadas para el caso peruano:

- `numItineraries: 12` · alternativas máx
- `transferPenalty: 1800s` · castigo por trasbordo
- `walkSpeed: 1.3 m/s` · velocidad caminata
- `boardCost: 600s` · costo de subir al bus
- `maxNumberOfTransfers: 4`

BRECHA CONOCIDA — PENDIENTE

El `router-config.json` aún **no** declara `gtfsRealtimeFeeds[]` apuntando al gtfs-rt-proxy. La integración existe (el proxy responde) pero falta cablear el polling oficial en la config de OTP.



`trufi-server-otp/Dockerfiles/2.8.1/config/otp-config.json`

Config completa en el repo upstream



`trufi-server-otp/Dockerfiles/2.8.1/config/router-config.json`

Todos los parámetros de routing



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





CÓMO SE LE PREGUNTA A OTP

Una consulta GraphQL paso a paso

Toda la app móvil habla con OTP por **una sola URL** POST con un body GraphQL. El cliente pide exactamente los campos que necesita y OTP responde solo con esos. Esto es lo que se ve cada vez que un usuario planifica un viaje.

Request – POST /otp/routers/default/index/graphql

GraphQL

```
{
  plan(
    from: { lat: -8.1116, lon: -79.0288 },
    to:   { lat: -8.0900, lon: -79.0500 },
    transportModes: [
      { mode: TRANSIT },
      { mode: WALK }
    ],
  ),
  numItineraries: 3
) {
  itineraries {
    duration
    walkDistance
    legs {
      mode
      startTime
      from { name }
      to   { name }
      route { shortName longName }
    }
  }
}
```

Response (recortada)

JSON

```
{
  "data": {
    "plan": {
      "itineraries": [{
        "duration": 1830,
        "walkDistance": 412.5,
        "legs": [
          {
            "mode": "WALK",
            "from": { "name": "Origin" },
            "to":   { "name": "Av. España" }
          },
          {
            "mode": "BUS",
            "startTime": 1714312800000,
            "route": {
              "shortName": "19",
              "longName": "Centro - El Porvenir"
            },
            "from": { "name": "Av. España" },
            "to":   { "name": "Av. Larco" }
          }
        ], ...]
      }
    }
  }
}
```

POR QUÉ GRAPHQL Y NO REST

El cliente decide qué campos quiere — la app móvil ahorra ancho de banda pidiendo solo lo que necesita. Una versión nueva de la app puede pedir más campos sin romper la API. OTP no necesita versionado de endpoints.



OpenTripPlanner – APIs

Documentación oficial del esquema GraphQL



Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - German International
Cooperation





SERVICIO · GTFS-RT-PROXY :8001

El puente entre el driver app y OTP

Python 3.11 + Flask + Gunicorn. Lee el feed protobuf que el driver app publica a S3, le inyecta el `trip_id` faltante, y lo entrega listo para OTP.

Cache 5s. Polling cada 30s desde OTP. 2 workers. ~75 LOC en `app.py`.

🔗 ENDPOINTS

- `GET /` — modo (sim/prod), upstream URL
- `GET /gtfsrt.proto` — feed enriquecido (binario)
- `GET /healthz` — JSON con stats del último ciclo

🇵🇪 STATS DEVUELTAS EN /HEALTHZ

```
{
  "status": "ok",
  "schedules_loaded": 247,
  "last_cache_age_s": 2.34,
  "last_stats": {
    "entities": 3,
    "rewritten": 3,
    "skipped_no_schedule": 0,
    "skipped_already_has_trip_id": 0,
    "entities_without_trip": 0
  }
}
```

⚙️ VARIABLES DE ENTORNO

```
SIMULATED=false
UPSTREAM_URL_PROD=https://trujillo-admin-storage
                        .s3.us-east-1.amazonaws.com/
                        driver/gtfsrt/pe-trujillo.proto
UPSTREAM_URL_SIM=http://gtfs-rt-simulator:9000
                        /pe-trujillo.proto
GTFS_PATH=/app/data/trujillo.gtfs.zip
CACHE_TTL=5
UPSTREAM_TIMEOUT=10
```

🛡️ CACHE BUSTING

El proxy fuerza `Cache-Control: no-cache` + un parámetro `?_=<ms>` al fetchear S3, para evitar leer un proto rancio del CDN.

📦 STACK

```
# requirements.txt
flask==3.0.3
gunicorn==22.0.0
requests==2.32.3
gtfs-realtime-bindings==1.0.0
protobuf<4.21,>4.21
```

```
# Dockerfile
python:3.11-slim
gunicorn --workers 2 --timeout 30
         --bind 0.0.0.0:8000
         --access-logfile -
```

⚠️ NOTA

Si `fetch + enrich` > 25s, el worker timeout (30s) lo mata. En condiciones normales son ~50ms.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale Zusammenarbeit





CÓMO FUNCIONA EL PROXY POR DENTRO

enrich() — qué pasa cuando llega un feed

Para cada vehículo en el feed, el proxy decide qué hacer. La mayoría de las veces, le inyecta el `trip_id` que falta. Cinco pasos.

1

¿Es un vehículo?

El feed puede tener entidades de varios tipos. Si no es vehículo, lo ignora.

2

¿Ya tiene `trip_id`?

Si sí, no hay nada que enriquecer — pasa de largo.

3

¿Tiene los datos para matchear?

Necesita `route_id` + `direction_id` + `start_time`. Sin alguno, no se puede.

4

Buscar en el GTFS

Llama a `schedule.match()` que cruza con el GTFS estático y devuelve el mejor candidato.

5

Reescribir + backfill

Inyecta el `trip_id`, alinea `start_time`, y completa el id del vehículo con la placa si faltaba.

LO QUE TIENES QUE RECORDAR

Los **stats** que devuelve `/healthz` te dicen cómo está yendo el match: si `skipped_no_schedule` sube, hay desfase entre el feed del driver y el GTFS estático.



`server/gtfs-rt-proxy/app.py`

Código completo · ~75 líneas



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





CÓMO SE ENCUENTRA EL TRIP_ID CORRECTO

El GTFS declara frecuencias. El proxy busca el slot más cercano.

El GTFS estático dice "cada 5 minutos sale un bus en la línea 19". El driver app reporta una hora aproximada de salida. La función `match()` encuentra el slot de frecuencia más cercano y devuelve el `trip_id` exacto.

1

Indexar al arrancar

Lee `trips.txt` + `frecuencias.txt` del GTFS y construye una tabla `(route_id, direction) → lista de slots`.

2

Buscar candidatos

Para cada vehículo, usa `bisect` (búsqueda binaria, `0(log n)`) sobre los slots del día.

3

Elegir el más cercano

Compara el slot anterior y el siguiente al driver-time, elige el de menor diferencia absoluta.

4

Devolver trip_id expandido

Concatena `base_trip_id` + índice del slot a 6 dígitos. Ejemplo: `"trip_19_1" + "000037"`.

REQUISITO CRÍTICO DEL GTFS

El GTFS del proxy **debe** tener `frecuencias.txt`. Si está expandido a `stop_times.txt`, el constructor lanza `RuntimeError`. Por eso hay dos GTFS distintos: el del proxy (1 MB, con frecuencias) y el de OTP (18 MB, expandido).



`server/gtfs-rt-proxy/schedule.py`

Implementación del lookup · ~80 líneas



Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





SERVICIO · GTFS-RT-SIMULATOR : 9000

Cuando no hay conductores, este genera un feed sintético

Threading HTTP server en Python que reproduce **trayectorias reales recordadas**, en loop, con interpolación lineal entre puntos. Útil para dev, presentaciones, fines de semana sin actividad real.

PIPELINE

```
# ONE-TIME (cuando se grabó historia):
GTFS-RT/history/*.proto
→ analyze.py
→ patterns.json (~260 KB, commiteado)

# RUNTIME (en el container):
patterns.json
→ server.py
→ build_pool(clone_factor)
→ build_feed(now)
→ /pe-trujillo.proto
```

DATOS QUE TIENE PATTERNS.JSON

```
{
  "trajectories_kept": 18,
  "trajectories": [
    {
      "route_id": "19",
      "direction_id": 1,
      "duration_seconds": 1125,
      "sample_plate": "PUN-7170",
      "points": [
        {"t": 0, "lat": -8.139, "lon": -79.06, "heading": 127.5},
        {"t": 5, "lat": -8.139, "lon": -79.06, "heading": 127.5},
        ...
      ]
    }
  ]
}
```

VARIABLES DEL SIMULADOR

- `CLONE_FACTOR=3` — clones por trayectoria
- `MAX_VEHICLES=0` — 0 = sin tope
- `PATTERNS_PATH=/app/patterns.json`
- `PORT=9000`

STAGGER DE CLONES

Cada clon arranca con un offset aleatorio dentro de su `duration_seconds` — así no aparecen todos juntos en el mismo punto. Seed fija: `42`.

LOOP INFINITO

```
elapsed = now - spawn_ts
loop_idx = int(elapsed // duration)
t_in_traj = elapsed - loop_idx * duration
# entity_id = "PUN-7170-3" (loop 3)
```

ENDPOINTS

- `GET /pe-trujillo.proto` — feed
- `GET /gtfsrt.proto` — alias
- `GET /healthz` — pool size, start time

ACTIVAR EL SIMULADOR

```
# En server/gtfs-rt-proxy/.env:
SIMULATED=true
CLONE_FACTOR=3
MAX_VEHICLES=10
```

`docker compose restart proxy`

REGENERAR PATTERNS.JSON

```
cd server/gtfs-rt-proxy/simulator
python analyze.py \
  --history ../../GTFS-RT/history \
  --out patterns.json \
  --min-duration 60 \
  --sample-interval 5
```

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale Zusammenarbeit





SERVICIO · PHOTON :2322

Geocoder de direcciones (Komoot Photon 0.4.2)

Convierte texto libre como "av. España 1234" en coordenadas. Backend: Java 21 sobre Alpine + Elasticsearch interno. Índice precomputado de Perú descargado de GraphHopper. **Sin gpu, sin training** — solo búsqueda full-text sobre OSM.

🔑 API — EJEMPLOS REALES

```
# Búsqueda
curl 'http://localhost:2322/api?q=plaza+armas+trujillo&limit=5'

{
  "features": [{
    "geometry": {
      "coordinates": [-79.0287, -8.1118],
      "type": "Point"
    },
    "properties": {
      "name": "Plaza de Armas",
      "city": "Trujillo",
      "country": "Peru",
      "type": "tourism"
    }
  }, ...]
}

# Reverse (coords → dirección)
curl 'http://localhost:2322/reverse?lat=-8.11&lon=-79.03'
```

🌐 PAÍSES DISPONIBLES (ISO-2)

El `init.sh` descarga la lista de países desde `download1.graphhopper.com/public/extracts/by-country-code/` y deja elegir. Para Trujillo: `pe`. Otros usados en el ecosistema Trufi: `bo`, `ec`, `rw`, `us`, `br`.

📦 DATOS — QUÉ BAJA INIT.SH

```
URL: https://download1.graphhopper.com/public/
extracts/by-country-code/pe/
photon-db-pe-latest.tar.bz2

# Tamaño aprox: ~varios GB (no commiteable)
# Se extrae a: /app/photon_data/elasticsearch/

# Estructura interna:
photon_data/
├─ elasticsearch/
│   └─ nodes/
│   └─ indices/
```

📁 MEMORIA

TAMAÑO PAÍS	RAM SUGERIDA
Perú, Bolivia, Ecuador	1-2 GB
Brasil, EEUU	4-8 GB

🔧 HEALTHCHECK

```
test: ["CMD", "wget", "--spider",
"http://localhost:2322/api?q=test"]
interval: 30s · start_period: 60s
```

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





SERVICIO · TILESERVER GL

Mapas vectoriales con 6 estilos listos

Imagen `maptiler/tileservver-gl:latest`. Sirve **vector tiles** (Mapbox Vector Tiles dentro de un .mbtiles, que es SQLite). La app móvil los renderiza con MapLibre GL en cliente — el servidor solo entrega trozos.

ESTILOS DISPONIBLES

- `osm-bright` — default · claro y detallado
- `osm-liberty` — Mapbox-like libre
- `maptiler-basic` — minimalista
- `positron` — gris claro para data viz
- `dark-matter` — noche / dashboards
- `fiord-color` — azul oscuro elegante

Cada estilo es un folder con `style.json` (Mapbox Style Spec v8) + sprites + glyphs.

API — PATRONES DE URL

```
# Tiles vectoriales
GET /data/{source}/{z}/{x}/{y}.pbf

# Style JSON (lo que carga MapLibre)
GET /styles/osm-bright/style.json

# Tiles raster (fallback)
GET /styles/osm-bright/{z}/{x}/{y}.png

# TileJSON
GET /data/openmaptiles.json

# Health
GET /health
```

CONFIG.JSON (GENERADO POR INIT.SH)

```
{
  "options": {
    "paths": {
      "root": "/data",
      "fonts": "fonts",
      "styles": "styles",
      "mbtiles": ""
    }
  },
  "data": {
    "openmaptiles": {
      "mbtiles": "/data/trujillo.mbtiles"
    }
  },
  "styles": {
    "osm-bright": {
      "style": "osm-bright/style.json",
      "tilejson": {
        "type": "overlay",
        "bounds": [-79.10, -8.20,
                  -78.96, -8.05]
      }
    }
  }
}
```

BBOX de Trujillo: `-79.10,-8.20,-78.96,-8.05` · Cómo se genera el .mbtiles: con `trufi-mbtiles-generator` desde el OSM extract de Perú.

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





SERVICIO · TRUFI-SERVER / YARP :88 :443

El frente: TLS + routing + analytics

.NET 10 (preview) + YARP 2.2.0 + LettuceEncrypt 1.3.3. Termina TLS automático con Let's Encrypt, enruta requests a los containers internos, y registra **cada** request en PostgreSQL para analítica posterior.

STACK INTERNO

- Imagen base: `aspnet:10.0-preview`
- `Yarp.ReverseProxy` 2.2.0
- `LettuceEncrypt` 1.3.3
- `EntityFrameworkCore` 10 + `Npgsql`
- PostgreSQL 16-alpine
- Channels para queue async

SSL AUTOMÁTICO

El proxy llama `AddLettuceEncrypt()` y la librería se encarga del resto: HTTP-01 challenge, emisión, renovación, persistencia.

- Configuración en `appsettings.json`: dominios + email
- Persistencia de certs en `/etc/lettuce-encrypt`
- Renovación automática 30 días antes

ROUTING 100% CONFIG-DRIVEN

Una sola línea en `Program.cs`:
`AddReverseProxy().LoadFromConfig(...)`. Toda la lógica vive en `appsettings.json`, que `setup.sh` modifica.

El mount es `:ro`: editar el JSON requiere `docker compose restart`.

LO QUE HAY QUE RECORDAR

Para agregar un nuevo servicio detrás del proxy, no se toca código — solo se corre `setup.sh` que muta el `appsettings.json` y se reinicia el contenedor.



trufi-association/trufi-server
Repo del proxy + analytics



server/docs/services/yarp.md
Doc operativa local



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





YARP · ANALYTICS

Cada request queda registrada en PostgreSQL

Middleware que **no bloquea**: encola la request en un `Channel` de System.Threading; un worker en background hace batches de 100 (o cada 1s) e inserta. Compresión auto-decodificada. Payload >1 MB: stub. Bytes nulos sanitizados (compatibilidad UTF-8 PostgreSQL).

ESQUEMA DE LA TABLA REQUESTS

COLUMNA	TIPO
id	bigint PK
method	varchar(10)
uri	text
host	varchar(255) (idx)
ip	varchar(45)
device_id	varchar(255) (idx)
user_agent	text
request_content_type	varchar(255) (idx)
request_headers	text (JSON)
body	text
status_code	int
response_content_type	varchar(255)
response_headers	text (JSON)
response_body	text
received_at	timestamp (idx)
created_at	timestamp

ANALYTICS API EXPUESTA EN /ANALYTICS-API/★

- `GET /logs` — listado con filtros (host, método, status, IP, fechas, búsqueda)
- `GET /stats` — totales: requests, dispositivos únicos, hosts únicos
- `GET /stats/hourly` — buckets por hora UTC
- `GET /stats/endpoints` — top URIs por volumen
- `GET /stats/devices` — top dispositivos
- `GET /swagger` — UI interactiva para probar

ROBUSTEZ DEL MIDDLEWARE

- Async vía `Channel<T>` — nunca bloquea la request
- Batch de 100 inserts o flush cada 1s
- Auto-decompresión de gzip / deflate / brotli
- Payloads > 1 MB: solo se guarda metadata
- Bytes nulos sanitizados (PostgreSQL UTF-8)

PARA EL DÍA A DÍA DEL ADMIN

Las queries SQL más usadas (top hosts, errores 5xx, dispositivos únicos) están en el **cheat sheet**. Para investigaciones puntuales, abrir Swagger en el browser y filtrar visualmente.



Gateway.Analytics

Middleware + controllers + payload helpers



Gateway.Shared/Models/Request.cs

El modelo EF que define la tabla



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz





YARP · SETUP.SH

Cómo se agrega un servicio nuevo al proxy

El script lee `trufi-proxy.json` del nuevo servicio, pide el dominio público, y muta `data/config/appsettings.json` con `jq`: agrega cluster YARP, ruta con match por hostname, y dominio a LettuceEncrypt para que renueve cert automáticamente.

1

El servicio define `trufi-proxy.json`

Con `name`, `container`, `port` y si quiere o no analytics.

2

Se ejecuta `./setup.sh`

Menú interactivo. Opción 1 pide la ruta del repo y el dominio público.

3

El script muta el JSON con `jq`

Agrega el dominio a LettuceEncrypt, define la ruta YARP, define el cluster con la dirección interna.

4

Reiniciar el contenedor

`docker compose restart server` recarga la config. La DB no se toca.

📄 MENÚ DE SETUP.SH

- [1] Agregar servicio externo
- [2] Exponer Analytics API
- [3] Listar servicios
- [4] Remover servicio
- [5] Cambiar email

📁 VOLUMES PERSISTENTES

- `data/config/` — `appsettings.json`
- `data/lettuce-encrypt/` — `certs`
- `data/postgres/` — `DB`

🔑 SCHEMA DE TRUFI-PROXY.JSON

- `name`, `description`
- `container`, `port`
- `analytics` (bool)



`trufi-server/setup.sh`

Script completo, ~430 líneas



`trufi-proxy.example.json`

Plantilla del schema



Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - German International
Cooperation



Embajada de Suiza en el Perú
Secretaría de Estado para Asuntos Externos SECO



► DEMO EN VIVO • ~12 MIN

Inspeccionemos el tráfico real con Swagger

YARP logea todas las peticiones. Vamos a abrir el Swagger UI de la Analytics API y a explorar la data en producción: top hosts, errores recientes, dispositivos únicos.

- 01 Abrir `https://trujillo.trufi.dev/analytics-api/swagger` en el browser
- 02 Probar `GET /stats` — total requests / dispositivos / hosts
- 03 Probar `GET /stats/hourly` — distribución por hora
- 04 Probar `GET /stats/endpoints?limit=10` — top URIs
- 05 Filtrar logs por errores: `GET /logs?statusCode=500`
- 06 Filtrar por dispositivo: `GET /logs?deviceId=...`
- 07 Comparar con SQL directo en la DB

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





SERVICIO · WEB-LANDING

La cara pública en trujillo.trufi.dev

Página estática con HTML + CSS + JS vanilla. Sin frameworks. nginx:alpine la sirve directo. i18n con JSON + `data-i18n`, theme toggle persistente, accesibilidad básica.

ESTRUCTURA DEL HTML

```
<header>
  logo + brand
  + langToggle (ES/EN)
  + themeToggle (sun/moon)

<section id="hero">
  h1 (con grad text)
  + 4 bullet features
  + store badges
    (Google Play / App Store)

<section id="alianzas">
  5 logos de socios

<footer>
  copyright + email + Trufi link
```

I18N - 14 KEYS

```
<!-- HTML -->
<p data-i18n="heroSub">...</p>
<span data-i18n-html="heroTitle">...</span>

// JS vanilla (140 LOC)
const lang = navigator.language.slice(0,2);
const dict = await fetch(`l10n/${lang}.json`)
  .then(r => r.json());
document.querySelectorAll('[data-i18n]')
  .forEach(el => el.textContent = dict[
    el.dataset.i18n
  ]);

// localStorage para persistir
```

THEME TOGGLE

```
// 18 CSS vars en :root
// + override en [data-theme="dark"]

// JS:
const saved = localStorage.getItem('theme');
const auto = matchMedia(
  '(prefers-color-scheme: dark)'
).matches ? 'dark' : 'light';
document.documentElement
  .dataset.theme = saved || auto;
```

NGINX.CONF

```
server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;
  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





DATOS · SERVER/DATA/

Inputs canónicos del sistema

Solo los archivos críticos viven en el repo (~26 MB). Los derivados (graph de OTP, índice de Photon) los regeneran los servicios al arrancar.

ARCHIVO	TAMAÑO	SERVICIO	FUNCIÓN
<code>trujillo.gtfs.zip</code>	18 MB	OTP	Schedule completo expandido
<code>trujillo.osm.pbf</code>	2.3 MB	OTP	Extract OSM (red vial)
<code>trujillo.mbtiles</code>	4.7 MB	tileserver-gl	Vector tiles para la app
<code>trujillo.gtfs.zip</code> (small)	1 MB	gtfs-rt-proxy	GTFS con frecuencias (lookup)
<code>patterns.json</code>	260 KB	simulator	18 trayectorias recordadas

SINCRONIZAR CON EL SERVER

```
SERVER=root@trujillo.trufi.dev

# pull (server → repo)
scp $SERVER:/root/otp/data/trujillo.gtfs.zip \
  server/data/

# push (repo → server, destructivo)
scp server/data/trujillo.gtfs.zip \
  $SERVER:/root/otp/data/
```

LO QUE NO SE COMMITEA

- `graph.obj` (~20 MB) — OTP regenera
- `photon_data/` (varios GB) — GraphHopper
- Backups `.bak`
- Archivos kigali (legacy)



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



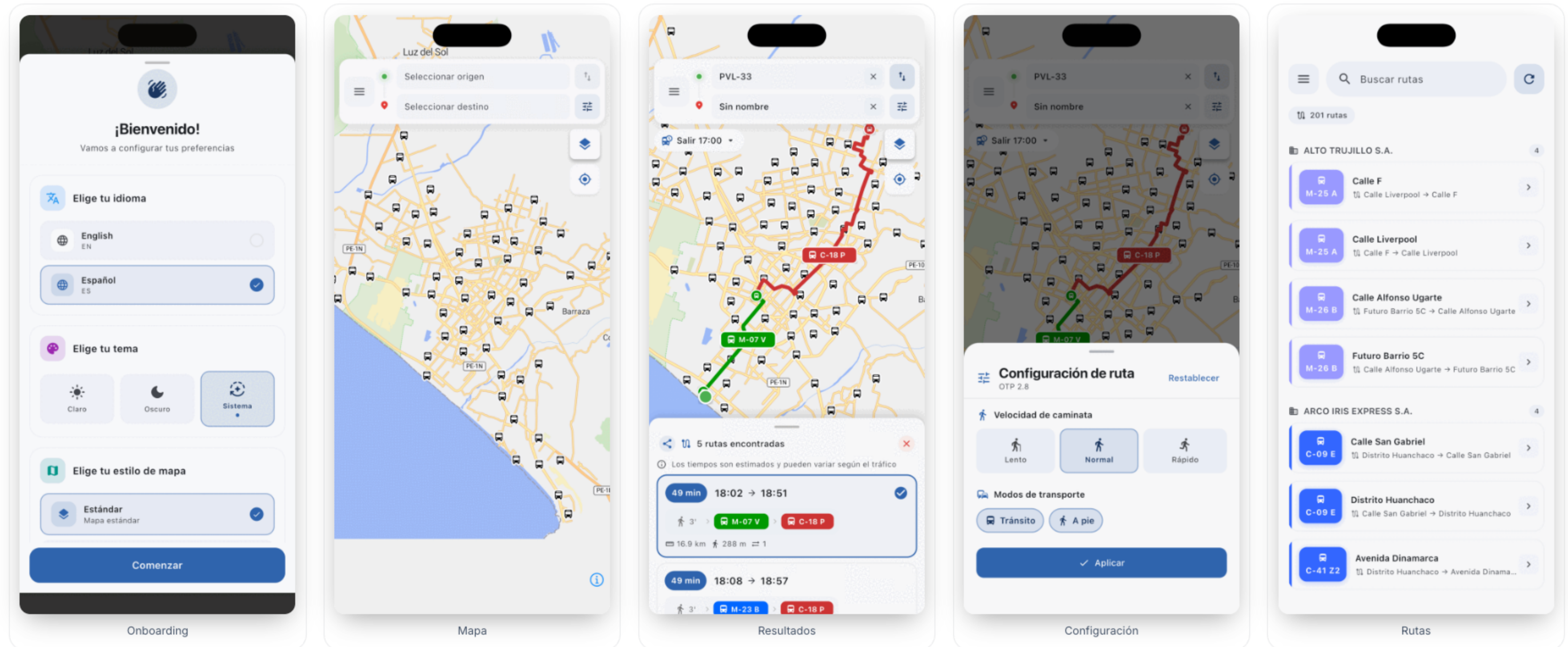
Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH



CLIENTE

App móvil MiRuta

Construida con **Flutter** sobre rutometro-app · iOS + Android · open source



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz





MOBILE · STACK Y CONFIGURACIÓN

Cómo está construida la app

Flutter 3.41.0 (locked vía FVM) sobre el framework **Trufi Core v5.8.0**. Estado con BLoC, navegación con go_router, mapas con MapLibre GL. Sin tracking de terceros, sin Google Maps. Versión actual: `1.3.0+9`.

DEPENDENCIAS CLAVE

- `flutter_bloc` 9.1 — state management
- `provider` 6.1 — inyección de dependencias
- `go_router` 17 — navegación
- `maplibre_gl` 0.25 — mapas
- `h3_flutter` 0.7 — H3 para analytics
- `Trufi Core` 5.8 — framework base

Trufi Core viene de GitHub packages, no de pub.dev.

A QUÉ SE CONECTA

- `otp.trujillo.trufi.dev` — GraphQL routing
- `photon.trujillo.trufi.dev` — geocoding
- `analytics.trujillo.trufi.dev` — request logs
- Tiles: assets offline incluidos en el bundle

ASSETS BUNDLEADOS

- `assets/routing/trujillo.gtfs.zip`
- `assets/offline/trujillo.mbtils`
- `assets/branding/*.png`

LOCALIZACIÓN

- `app_es.arb` — Español (default)
- `app_en.arb` — English
- `app_de.arb` — Deutsch (fallback)
- ~50 strings por idioma

VERSIÓN

`1.3.0+9` · iOS + Android · open source

⚠ IMPORTANTE PARA EL ADMIN

Toda la configuración (URLs del backend, ciudad, coordenadas, colores, contacto) está **hardcodeada** en `lib/main.dart`. No hay `.env`. Cambiar cualquier cosa implica editar el archivo y compilar una nueva versión.



trufi-association/rutometro-app

Repo upstream de la app



mobile-app/lib/main.dart

Toda la configuración del cliente



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





MOBILE · BUILD & RELEASE

Cómo se compila y publica una nueva versión

No hay CI configurado — los builds son manuales. FVM es obligatorio (Flutter 3.41.0 está pineado). Detalle completo en [mobile-app/docs/BUILD_GUIDE.md](#) y [mobile-app/docs/RELEASE_GUIDE.md](#).

1

Setup local (una vez)

Instalar FVM, instalar Flutter 3.41.0, ejecutar `fvm flutter pub get`. Sin FVM las dependencias rompen.

2

Subir versión

Editar `pubspec.yaml` incrementando el build number (después del `+`).

3

Compilar

Android: `build appbundle` (.aab para Play Store). iOS: `build ios --no-codesign`.

4

Firmar (Android)

Requiere `android/key.properties` + `upload-keystore.jks` (no se commitean).

5

Firmar (iOS)

Abrir `Runner.xcworkspace`, configurar Team, `Product` → `Archive` → `Distribute`.

6

Publicar

Google Play Console (subir `.aab`) · App Store Connect (subir desde Xcode).

LO QUE VAS A OLVIDAR Y VA A DOLER

Si compilas con `flutter` directo en lugar de `fvm flutter`, las dependencias de Trufi Core se rompen silenciosamente. Si `android/key.properties` está mal configurado, el build falla silencioso — siempre usar `--verbose` ante dudas.



[mobile-app/docs/BUILD_GUIDE.md](#)
Guía de compilación paso a paso



[mobile-app/docs/RELEASE_GUIDE.md](#)
Proceso completo de release



Checklist antes de publicar: subir versión `version` en `pubspec.yaml` (siempre incrementar el build number después del `+`), commit + tag, build, smoke test en device real, upload, llenar release notes en la consola.

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - German Development Cooperation





MOBILE · TAREAS COMUNES

Lo que vas a tener que hacer en la app

Las 9 tareas más frecuentes que un admin ejecuta sobre la app móvil. Cada una es un cambio + rebuild + release. Detalle en las próximas slides.

RELEASES

- Subir versión (build number / semver)
- Hotfix urgente — fast publish

CONFIGURACIÓN

- Cambiar URL del backend
- Cambiar ciudad / coordenadas
- Actualizar GTFS / mapas offline

UI / CONTENIDO

- Logos / branding
- Agregar idioma nuevo
- Deep links (pendiente)
- Cosas que vas a olvidar



`mobile-app/lib/main.dart`

Toda la configuración hardcodeada



Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
giz - Gesellschaft für Internationale Zusammenarbeit





ADMIN · 1 / 9

Subir versión de la app

Cada vez que quieres publicar cambios, hay que incrementar el build number (siempre) y opcionalmente el semver (para features visibles).

EDITAR PUBSPEC.YAML

```
version: 1.3.0+10
#      ^^^^^ ^
#      semver build_number

# Solo bump de build (parche interno)
1.3.0+9 → 1.3.0+10

# Bump de patch (bug fix usuarios)
1.3.0+10 → 1.3.1+11

# Bump de minor (feature nueva)
1.3.0+10 → 1.4.0+11
```

TAG Y PUSH

```
git add pubspec.yaml
git commit -m "Bump 1.3.0+10 → 1.3.0+11"
git tag v1.3.0+11
git push --tags
```

El tag es importante para correlacionar crashes/issues con la versión exacta del binario.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





Cambiar la URL del backend

Las URLs de OTP y Photon están hardcodeadas en `lib/main.dart`. Cambiarlas requiere recompilar y publicar nueva versión.

LÍNEAS A EDITAR

```
// lib/main.dart

// línea 51 - URL de OTP
otpEndpoint:
  'https://trujillo.trufi.dev/otp/...',

// línea 221 - URL de Photon
photonEndpoint:
  'https://trujillo.trufi.dev/photon/...',
```

CONSIDERACIONES

- Los usuarios **actuales** seguirán usando la URL vieja hasta que actualicen la app
- Si la URL vieja deja de funcionar, los usuarios sin actualizar quedan sin servicio
- Considerar mantener un redirect en YARP por unas semanas
- Forzar update vía Play Store / App Store si el cambio es breaking



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





Actualizar GTFS o mapas offline en la app

La app embebe el GTFS y los vector tiles para funcionar sin conexión. Cuando cambia la red de transporte o el mapa, hay que reemplazar los assets y rebuildear.

ARCHIVOS A REEMPLAZAR

```
mobile-app/  
├── assets/  
│   ├── routing/  
│   │   └── trujillo.gtfs.zip  ← GTFS  
│   └── offline/  
│       └── trujillo.mbtiles  ← mapa
```

Sin código que tocar — solo reemplazar los archivos y recompilar.

⚠ CUIDADO

- Si `trujillo.gtfs.zip` falta, la app **crashea** al abrir `TransportListScreen`
- El zip debe ser válido — verificar con `unzip -t trujillo.gtfs.zip`
- El mbtiles puede subir el tamaño del binario rápido — comprimirlo con tilemaker si pasa los 10 MB
- Probar en device real antes de publicar — el emulador a veces tolera assets corruptos



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





ADMIN · 4 / 9

Cambiar logos y branding

El branding visible al usuario está repartido entre los assets de la app y los iconos del launcher de cada plataforma.

LOGOS EN PANTALLA

```
mobile-app/assets/branding/  
├─ app-logo.png      ← splash, drawer  
├─ miruta-icon.png   ← header  
└─ partners/        ← logos de socios  
    ├─ MPT-logo.png  
    ├─ MTC-logo.png  
    └─ ...
```

Reemplazar el archivo (mismo nombre y dimensiones).

ICONO DEL LAUNCHER

```
# Android (varios tamaños)  
android/app/src/main/res/  
├─ mipmap-mdpi/    ic_launcher.png (48px)  
├─ mipmap-hdpi/    ic_launcher.png (72px)  
├─ mipmap-xhdpi/   ic_launcher.png (96px)  
├─ mipmap-xxhdpi/  ic_launcher.png (144px)  
└─ mipmap-xxxhdpi/ ic_launcher.png (192px)  
  
# iOS (set completo)  
ios/Runner/Assets.xcassets/  
└─ AppIcon.appiconset/ (varios PNG)
```

Usar [flutter_launcher_icons](#) para generar todos los tamaños desde un único PNG fuente.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





ADMIN · 5 / 9

Cambiar ciudad / coordenadas

Si la app va a desplegarse en otra ciudad, hay que cambiar el centro del mapa, el nombre y — crítico — el bounding box que Photon usa para el geocoding.

CENTRO Y METADATA

```
// lib/main.dart líneas 31-34

defaultLatLng: LatLng(-8.108, -79.029),
defaultZoom: 13.0,
city: 'Trujillo',
countryCode: 'PE',
```

BBOX DE PHOTON (CRÍTICO)

```
// lib/main.dart línea 224

photonBbox: '-79.10,-8.20,-78.95,-8.05',
//           ^min_lon ^min_lat ^max_lon ^max_lat

// Photon devuelve 0 resultados FUERA de
// este bbox – si está mal el geocoding
// "no encuentra nada"
```

Sacar el bbox correcto desde bboxfinder.com dibujando un polígono sobre la ciudad.



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
giz
giz





ADMIN · 6 / 9

Agregar un idioma nuevo a la app

La i18n usa archivos ARB (Application Resource Bundle, formato Flutter). Hay ~50 keys que traducir. Añadir un idioma toma ~30 min de copiar/traducir/regenerar.

PASOS

```
# 1. Crear el ARB del idioma nuevo
cp lib/l10n/app_es.arb \
  lib/l10n/app_qu.arb    # quechua

# 2. Traducir las ~50 keys (cambiar @@locale)

# 3. Regenerar las clases Dart
fvm flutter gen-l10n

# 4. Recompilar
fvm flutter run
```

REGISTRAR EL LOCALE

```
// lib/main.dart

supportedLocales: [
  Locale('es'),
  Locale('en'),
  Locale('qu'),    // ← nuevo
],
```

El idioma del sistema del usuario manda — si no hay match exacto, cae al primero soportado (`es`).

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
giz - German International Cooperation





ADMIN · 7 / 9

Hotfix urgente en producción

Cuando la app está caída para usuarios reales y necesitas publicar una corrección lo más rápido posible. Tanto Play como App Store tienen mecanismos para acelerar.

FLUJO

```
# 1. Pull, fix, bump del patch
git pull
# (hacer el fix mínimo)
# pubspec.yaml: 1.3.0 → 1.3.1

# 2. Build release
fvm flutter build appbundle --release

# 3. Build iOS
fvm flutter build ipa --release
```

PUBLICAR RÁPIDO

- **Android — Play Console:** usar "Production · Fast publish" (review acelerado, 2-4 hs)
- **iOS — App Store Connect:** Archive + "Expedited review" (justificar el motivo). Aprueban en ~24 hs
- Mientras esperas revisión, considerar bajar la versión rota con "halt rollout" para evitar más users afectados



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
giz - Gesellschaft für Internationale Zusammenarbeit





ADMIN · 8 / 9 · PENDIENTE

Deep links — para que las URLs del sitio abran la app

Cuando un usuario clicka `trujillo.trufi.dev/route/19` en su browser, lo ideal es que se abra la app si la tiene instalada (con el contexto), en lugar del browser. Hoy esto NO está activo — falta publicar 2 archivos en la web.

ANDROID — ASSETLINKS.JSON

```
# Publicar en:
trujillo.trufi.dev/.well-known/assetlinks.json

# Contenido:
[{"relation": ["delegate_permission/common.handle_all_urls"],
 "target": {"namespace": "android_app",
  "package_name": "pe.trujillo.miruta",
  "sha256_cert_fingerprints": [
    "<SHA-256 del cert release>"
  ]
}
}]
```

IOS — APPLE-APP-SITE-ASSOCIATION

```
# Publicar en:
trujillo.trufi.dev/.well-known/apple-app-site-association

# Contenido (sin extensión .json):
{
  "applinks": {
    "apps": [],
    "details": [{
      "appID": "<TEAM_ID>.pe.trujillo.miruta",
      "paths": ["*"]
    }]
  }
}
```

El web-landing nginx debe servir estos paths con `Content-Type: application/json` y sin redirects.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - German Development Cooperation





ADMIN · 9 / 9 · GOTCHAS

Cosas que vas a olvidar

Detalles no obvios que cuestan tiempo cuando los descubris en producci3n. Memorizar antes de necesitarlos.

FVM ES OBLIGATORIO

Sin **FVM** (Flutter Version Manager) `trufi-core` rompe en build. Si llegas a un repo limpio, instala FVM antes de cualquier `flutter pub get`.

```
brew tap leoafarias/fvm
brew install fvm
fvm install
fvm use # respeta .fvmrc
```

MAPAS OFFLINE NO SE AUTO-ACTUALIZAN

El `.mbtiles` queda fijo en el binario. Si actualizas el mapa upstream, los usuarios no lo ven hasta el siguiente release.

SIN RETRY SI OTP CAE

El cliente HTTP de la app no reintenta. Si OTP devuelve un error, el usuario ve "no se encontraron rutas". Considerar agregar retry con backoff en alg3n momento.

HEADERS DE ANALYTICS

Cada request a OTP/Photon manda headers `X-Origin-*` custom (device_id, app_version, locale). YARP los loggea a la DB para los reportes.

```
X-Origin-DeviceId: a1b2c3...
X-Origin-AppVersion: 1.3.0+10
X-Origin-Locale: es
```

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
giz
giz





Cómo se opera el sistema día a día

Ubuntu 22.04 + Docker. 4 GB RAM (ojo: sin swap). Health check global, deploy por servicio, backups, troubleshooting común.

🔍 HEALTH CHECK GLOBAL

```
ssh root@trujillo.trufi.dev '
  docker ps --format "table {{.Names}}\t{{.Status}}"
  echo
  df -h | grep -v overlay
  free -h
  uptime
'
```

🚩 HEALTHCHECKS "FALSOS NEGATIVOS"

CONTAINER	ESTADO
server-server-1	/dev/tcp no existe en /bin/sh
trufi-tileserv-g1	healthcheck.js falla, GET / responde 200

📄 LOGS

```
docker logs -f <container>
docker logs --tail 100 <container>
docker logs --since 30m <container>
docker logs -t <container> # timestamps
```

🚀 DESPLIEGUE DE CAMBIOS

```
# Servicio local (proxy, web-landing):
git push # this repo
rsync -avz --exclude='.git' \
  ./server/<svc>/ \
  root@trujillo.trufi.dev:/root/<svc>/
ssh root@trujillo.trufi.dev \
  "cd /root/<svc> && docker compose up -d --build"

# Servicio submódulo (otp, photon, etc.):
ssh root@trujillo.trufi.dev "
  cd /root/<svc>
  git pull
  docker compose up -d --build
"
```

♻️ REFRESCAR GTFS

```
scp server/data/trujillo.gtfs.zip \
  root@trujillo.trufi.dev:/root/otp/data/
ssh root@trujillo.trufi.dev '
  rm /root/otp/data/graph.obj
  cd /root/otp && docker compose restart
'
docker logs -f otp # ~2-3 min
```

📁 BACKUPS

```
docker exec server-db-1 \
  pg_dump -U analytics analytics \
  > backups/analytics-$(date +%F).sql
```



CUANDO ALGO FALLA

Troubleshooting — qué hacer y por dónde empezar

Síntomas comunes con su diagnóstico y solución. Si ningún caso acá resuelve tu problema, los logs (`docker logs <container>`) casi siempre dan la pista. También ver los docs específicos de cada servicio en [server/docs/services/](#).

● CRÍTICOS (SERVICIO CAÍDO)

- 502 Bad Gateway en algún subdominio
- App no muestra ETAs / posiciones
- OTP no levanta / colgado en build
- No puedo conectarme por SSH

● DEGRADACIÓN / CAPACIDAD

- Container `unhealthy` pero funciona
- Memoria al límite (<200 MB free)
- Disco lleno (>90%)
- SSL no se renueva
- DB de analytics se pone lenta

📅 PARA REPORTAR INCIDENTES

Recolectar contexto en un solo comando para pegar en un ticket
— slide siguiente al final de la sección.

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
giz - Gesellschaft für Internationale Zusammenarbeit





TROUBLESHOOTING · 1 / 10 ·  DEGRADACIÓN

Container marcado **unhealthy** pero el servicio funciona

Healthchecks mal configurados que dan falsos negativos. El servicio responde pero Docker lo marca como roto.

CASOS CONOCIDOS

- `server-server-1` — usa `/dev/tcp` que no existe en su shell
- `trufti-tilserver-gl` — `node healthcheck.js` falla pero `GET /` responde 200

VERIFICAR QUE SÍ FUNCIONA

```
docker logs --tail 20 <container>
# si hay 200s saliendo, el servicio está bien
```

SOLUCIÓN

Dos opciones:

- **Ignorarlo** — si está documentado como falso negativo, no es prioridad
- **Arreglar el healthcheck** en el `docker-compose.yml` del servicio

```
# Ejemplo: cambiar el healthcheck
services:
  servicio:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/"]
      interval: 30s
      timeout: 10s
      retries: 3
```

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





TROUBLESHOOTING · 2 / 10 · ● CRÍTICO

502 Bad Gateway en algún subdominio

YARP recibió la request, intentó proxearla al backend interno (OTP, Photon, etc.) y el backend no respondió. Casi siempre el backend está caído o reiniciando.

DIAGNÓSTICO

```
# 1. ¿Qué container falta o está reiniciando?
docker ps -a

# 2. Ver logs del culpable
docker logs --tail 50 <container>

# 3. Verificar la red trufi-server
docker network inspect trufi-server
```

SOLUCIÓN

```
# Restart simple
docker restart <container>

# Si está realmente roto, recrear
cd /root/<svc>
docker compose down
docker compose up -d --build

# Verificar que YARP lo ve de nuevo
curl -I https://trujillo.trufi.dev/<path>
```



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





TROUBLESHOOTING · 3 / 10 · ● CRÍTICO

La app no muestra ETAs ni posiciones de buses

El feed GTFS-RT no está llegando a OTP, o llega pero los `trip_id` no matchean. Síntoma típico: el mapa carga, las rutas existen, pero los buses están "congelados" o no aparecen.

VERIFICAR EL PROXY

```
curl http://127.0.0.1:8001/healthz | jq

# Mirar last_stats:
# - rewritten alto → todo bien
# - skipped_no_schedule alto
#   → GTFS desactualizado
# - skipped_already_has_trip_id alto
#   → driver app cambió formato
# - HTTP error → S3 no responde
```

CAUSAS Y SOLUCIONES

- **GTFS desactualizado:** subir nuevo `trujillo.gtfs.zip` + restart proxy + restart OTP (regenerar graph)
- **S3 no responde:** verificar que `trujillo-admin-storage` esté accesible
- **Driver app caído:** contactar al equipo de driver
- **OTP no consume el feed:** revisar `otp-config.json` updaters

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





TROUBLESHOOTING · 4 / 10 · CAPACIDAD

Memoria al límite — `free -h` muestra < 200 MB libre

El droplet tiene 4 GB de RAM y **sin swap por defecto**. OTP rebuild + Photon corriendo + analytics DB se acercan al techo. Cualquier pico mata containers con OOM.

DIAGNÓSTICO

```
# Ver memoria total y swap
free -h

# Ver swap activo (si lo hay)
swapon --show

# Ver uso por container
docker stats --no-stream

# Procesos top por consumo de RAM
ps aux --sort=-%mem | head -10
```

CREAR SWAP POR PRIMERA VEZ (4 GB)

```
# 1. Crear el archivo (4 GB)
fallocate -l 4G /swapfile

# 2. Permisos solo root
chmod 600 /swapfile

# 3. Formatear como swap
mkswap /swapfile

# 4. Activar
swapon /swapfile

# 5. Verificar
free -h
swapon --show
```

HACER EL SWAP PERMANENTE

```
# Agregar a fstab para que monte al reboot
echo '/swapfile none swap sw 0 0' \
>> /etc/fstab

# Bajar swappiness (default 60 → 10)
# para que el kernel use swap solo
# cuando RAM se agota, no antes
echo 'vm.swappiness=10' \
>> /etc/sysctl.conf
sysctl -p
```

AUMENTAR SWAP EXISTENTE

```
# 1. Desactivar el actual
swapoff /swapfile

# 2. Recrear con tamaño nuevo (8 GB)
rm /swapfile
fallocate -l 8G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile

# fstab no se toca: la entrada
# es agnóstica al tamaño
```

A largo plazo: subir el droplet a 8 GB de RAM, o mover Photon a otra instancia (es el que más memoria consume).

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





TROUBLESHOOTING · 5 / 10 · CAPACIDAD

Disco lleno — `df -h` muestra > 90%

El droplet tiene 78 GB. Lo que más crece: imágenes Docker viejas, logs de containers, backups `.bak` sin rotación, y `photon_data/` (varios GB legítimos).

IDENTIFICAR CULPABLES

```
# Ver disco general
df -h | grep -v overlay

# Top 10 directorios más grandes
du -h /root/* | sort -hr | head -10

# Tamaño de imágenes Docker
docker images --format \
  "table {{.Repository}}\t{{.Size}}"
```

LIMPIEZA SEGURA

```
# Docker cleanup (imágenes y volúmenes)
docker system prune -a --volumes

# Backups viejos de OTP
rm /root/otp/*.bak.*
rm -rf /root/otp/old_data/

# Logs gigantes (con cuidado)
truncate -s 0 \
  $(docker inspect <container> \
    --format='{{.LogPath}}')
```

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





TROUBLESHOOTING · 6 / 10 · ● CRÍTICO

OTP no levanta o queda colgado en build del graph

OTP construye un graph en memoria desde GTFS + OSM al primer arranque. El proceso toma 2-3 minutos. Si tarda más o falla, los logs dicen exactamente por qué.

DIAGNÓSTICO – LEER LOGS

```
docker logs -f otp

# Errores típicos:
# "Could not find GTFS data"
#   → GTFS no está en /app/data/
# "OutOfMemoryError: Java heap space"
#   → subir JAVA_MAX_MEMORY o mem_limit
# "Building graph" + 5 min → normal, esperar
# "Loaded N stop times" → ya está casi listo
```

FORZAR REBUILD LIMPIO

```
cd /root/otp

# Borrar el graph viejo
rm data/graph.obj

# Reiniciar (regenera todo)
docker compose restart

# Mirar el progreso (~2-3 min)
docker logs -f otp
```

Si el OOM persiste, edita `JAVA_MAX_MEMORY=2g` en el compose.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





TROUBLESHOOTING · 7 / 10 · CAPACIDAD

SSL no se renueva — el candado en el browser desaparece

LettuceEncrypt renueva certificados 30 días antes de expirar. Si no pudo validar el dominio (HTTP-01 challenge), el certificado expira y los clientes ven advertencias de seguridad.

DIAGNÓSTICO

```
docker logs server-server-1 \
  | grep -i "lettuce\|encrypt\|cert"

# Ver fecha de expiración
echo | openssl s_client \
  -servername trujillo.trufi.dev \
  -connect trujillo.trufi.dev:443 2>/dev/null \
  | openssl x509 -noout -dates
```

CAUSAS COMUNES

- **Puerto 80 bloqueado** — LettuceEncrypt valida vía HTTP-01, necesita el :80 abierto
- **DNS no apunta al server** — verificar A record
- **Email mal configurado** en `appsettings.json` de YARP
- **Rate limit de Let's Encrypt** — esperar 1h o usar staging

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





TROUBLESHOOTING · 8 / 10 · ● CRÍTICO

No puedo conectarme por SSH

Si no puedes entrar al server, no puedes diagnosticar nada más. Hay 3 causas típicas para descartar antes de pánico.

CHECKLIST

- ¿Tu llave pública está en `/root/.ssh/authorized_keys`? Pedir a otro admin que verifique
- ¿El droplet está arriba? — entrar al dashboard de DigitalOcean
- ¿El servicio SSH está corriendo? — desde la consola web del droplet
- ¿Cambió la huella del host? (ssh muestra warning) — revisar `~/.ssh/known_hosts`

El servidor actualmente **no tiene fail2ban**, así que el banneo automático por intentos fallidos no aplica.

COMANDOS ÚTILES

```
# Verbose mode — muestra qué llave intenta
ssh -v root@trujillo.trufi.dev

# Si tienes acceso por consola web (D0):
systemctl status ssh
journalctl -u ssh --since "1 hour ago"

# Listar llaves activas en el server
cat /root/.ssh/authorized_keys

# Si la huella del host cambió:
ssh-keygen -R trujillo.trufi.dev
```

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale Zusammenarbeit





TROUBLESHOOTING · 9 / 10 · CAPACIDAD

La DB de analytics se pone lenta

YARP loggea cada request a la tabla `requests` de PostgreSQL. Sin rotación, en pocos meses pasa los 100M de filas y las queries se vuelven lentas.

VERIFICAR TAMAÑO

```
docker exec -it server-db-1 \
  psql -U analytics -d analytics

-- Tamaño total de la tabla
SELECT pg_size_pretty(
  pg_total_relation_size('requests'));

-- Cantidad de filas
SELECT count(*) FROM requests;

-- Top device_ids
SELECT device_id, count(*)
  FROM requests
 GROUP BY device_id
 ORDER BY 2 DESC LIMIT 10;
```

LIMPIEZA PERIÓDICA

```
-- Borrar requests > 30 días
DELETE FROM requests
  WHERE received_at
         < now() - interval '30 days';

-- Recuperar espacio físico
VACUUM ANALYZE requests;

-- Si quedó muy fragmentada (último recurso)
VACUUM FULL requests;
-- ↑ bloquea la tabla, hacerlo en hora baja
```

A largo plazo: agregar un cronjob diario que borre > N días.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





Recolectar contexto en un solo comando

Cuando vas a reportar un incidente (a Trufi, al equipo, a un ticket), conviene incluir un snapshot del estado del server. Este one-liner trae todo lo importante en formato copiable.

SNAPSHOT COMPLETO

```
ssh root@trujillo.trufi.dev '
echo "==== CONTAINERS ===="
docker ps --format \
  "table {{.Names}}\t{{.Status}}"

echo
echo "==== DISK ===="
df -h | grep -v overlay

echo
echo "==== MEMORY ===="
free -h

echo
echo "==== UPTIME ===="
uptime

echo
echo "==== PROXY HEALTH ===="
curl -s http://127.0.0.1:8001/healthz \
  | head -30
'
```

CÓMO USARLO

Copia la salida y pégala en:

- El issue/ticket donde reportas el problema
- El canal Slack/Telegram del equipo
- Un mensaje al mantenedor de Trufi si es algo upstream

El comando es read-only — no modifica nada del server, puedes ejecutarlo siempre.



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale Zusammenarbeit





Conectarse, panorama y logs

Lo primero que vas a escribir cada vez que abres una sesión. Después de SSH, este es el bloque que te dice "¿el sistema está vivo?" y "¿qué dijo X servicio?".

CONECTARSE + PANORAMA

```
ssh root@trujillo.trufi.dev

# Health check completo en una línea:
docker ps --format \
  "table {{.Names}}\t{{.Status}}"
df -h | grep -v overlay
free -h && uptime
```

VER LOGS DE UN SERVICIO

```
docker logs <container>          # ver
docker logs -f <container>       # tail -f
docker logs --tail 100 <container> # últimos N
docker logs --since 30m <container>
docker logs -t <container>       # timestamps

# Logs filtrados
docker logs <c> 2>&1 | grep -i error
```




CHEAT SHEET · 2 / 5 · DEPLOY

Reiniciar / actualizar servicios + submódulos

El loop de cambios pequeños: editar config, reiniciar el servicio. Y para servicios upstream (OTP, Photon, etc.), pull del submódulo y rebuild.

REINICIAR / ACTUALIZAR UN SERVICIO

```
cd /root/<servicio>

# Reiniciar (sin tocar imagen)
docker compose restart

# Actualizar (rebuild de imagen)
docker compose up -d --build

# Bajar
docker compose down

# Ver compose actual con env resuelto
docker compose config
```

SUBMÓDULOS (OTP, PHOTON, ...)

```
# Update a último main de un solo svc
cd /root/otp
git pull origin main
docker compose up -d --build

# Update todos los submódulos (en local)
cd ~/workspace/trujillo
git submodule update --remote --merge
git commit -am "bump submodules"
git push
```

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
giz
GIZ
DEUTSCHE ZUSAMMENARBEIT



Schweizerische Eidgenossenschaft
Confédération suisse
Confederazione Svizzera
Confederaziun svizra
Embajada de Suiza en el Perú
Secretaría de Estado para Asuntos Externos SECO





CHEAT SHEET · 3 / 5 · DATA

Refrescar data + backup de la DB

Subir/bajar GTFS y mapas entre tu repo local y el server, y dump/restore de la base de analytics.

REFRESCAR DATA DEL SERVER

```
SERVER=root@trujillo.trufi.dev

# Bajar al repo (después de cambios en server)
scp $SERVER:/root/otp/data/trujillo.gtfs.zip \
  server/data/

# Subir al server (destructivo)
scp server/data/trujillo.gtfs.zip \
  $SERVER:/root/otp/data/

# Si subiste GTFS: regenerar graph en OTP
ssh $SERVER 'rm /root/otp/data/graph.obj && \
  cd /root/otp && docker compose restart'
```

BACKUP DE LA DB

```
# Dump
docker exec server-db-1 \
  pg_dump -U analytics analytics \
  > backups/db-$(date +%F).sql

# Restaurar
cat backups/db-X.sql | docker exec -i \
  server-db-1 psql -U analytics -d analytics

# Sincronizar a tu máquina (rsync)
rsync -avz $SERVER:backups/ ./backups/
```

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





Probar APIs desde el server + limpieza

Cuando algo no funciona desde afuera, primero verificar si funciona desde adentro. Y cuando el disco se queja, comandos de limpieza seguros.

PROBAR APIS DESDE EL SERVER

```
# Health del proxy
curl http://127.0.0.1:8001/healthz | jq

# OTP responde?
curl -I http://127.0.0.1:8080/otp/

# Photon (geocoding test)
curl 'http://localhost:2322/api?q=trujillo'

# YARP / sitio público (desde fuera)
curl -I https://trujillo.trufi.dev

# Headers de respuesta de YARP
curl -sI https://trujillo.trufi.dev/otp/
```

LIMPIEZA PERIÓDICA

```
# Imágenes Docker viejas
docker image prune -a

# Containers, redes, volumes huérfanos
docker system prune --volumes

# Logs viejos de un container
truncate -s 0 \
  $(docker inspect -f '{{.LogPath}}' \
    --format='{{.LogPath}}')

# Backups con > 30 días
find backups/ -mtime +30 -delete
```



Despliegue full + SQL útil sobre analytics

El flujo de despliegue de un cambio (de tu máquina al server), y queries que vas a usar para revisar tráfico y errores en la DB.

DESPLIEGUE FULL

```
# 1. desde tu repo local
git push

# 2. en server: pull + rebuild
ssh root@trujillo.trufi.dev '
  cd /root/<svc>
  git pull
  docker compose up -d --build
'

# 3. verificar
docker logs -f <svc>

# 4. test desde fuera
curl -I https://trujillo.trufi.dev/<path>
```

SQL ÚTIL SOBRE ANALYTICS

```
docker exec -it server-db-1 \
  psql -U analytics -d analytics

-- Top hosts última hora
SELECT host, COUNT(*) FROM requests
WHERE created_at > now() - interval '1 hour'
GROUP BY host ORDER BY 2 DESC LIMIT 10;

-- Errores 5xx recientes
SELECT uri FROM requests
WHERE status_code ≥ 500
ORDER BY received_at DESC LIMIT 20;

-- Top devices únicos hoy
SELECT COUNT(DISTINCT device_id)
FROM requests
WHERE received_at > CURRENT_DATE;
```




Cómo agregar y remover admins del sistema

Cuando entra alguien nuevo al equipo, o sale, tienes que actualizar las llaves SSH. **Nunca compartas tu llave privada** — cada persona usa la suya.

+ AGREGAR UN NUEVO ADMIN

El nuevo admin te manda su clave **pública** (el archivo `.pub`):

```
# En el server (tú, admin actual):
ssh root@trujillo.trufi.dev

# Editar authorized_keys
nano /root/.ssh/authorized_keys

# Pegar la línea que te mandó al final:
ssh-ed25519 AAAA... usuario@laptop

# Guardar (Ctrl+O, Enter, Ctrl+X)

# Verificar permisos correctos:
chmod 600 /root/.ssh/authorized_keys
```

Pedile que pruebe inmediatamente después.

— REMOVER UN ADMIN

Identifica la línea con el comentario al final (email/usuario):

```
# Ver llaves actuales con info
cat /root/.ssh/authorized_keys

# Cada línea termina con un comentario tipo:
# ssh-ed25519 AAAA... pepe@laptop-vieja

# Borrar la línea con el editor
nano /root/.ssh/authorized_keys
# (borrar la línea entera)
```

🔒 BUENAS PRÁCTICAS

- Cada admin tiene **su** par de llaves
- El comentario debe identificar a la persona (email)
- Rotar llaves anualmente o ante incidente
- Backup de `authorized_keys` en lugar seguro
- **Nunca** commitear llaves privadas en git

📅 AUDITAR QUIÉN TIENE ACCESO

```
# Listar todas las llaves activas
cat /root/.ssh/authorized_keys

# Contar cuántas hay
wc -l /root/.ssh/authorized_keys

# Ver últimos logins
last -F | head -20

# Logs de SSH (intentos fallidos)
journalctl -u ssh --since "1 day ago" \
| grep "Accepted\\|Failed"
```

🚨 SI UNA LLAVE SE FILTRÓ

1. Borrar la línea de inmediato
2. Pedirle a la persona que regenere su par
3. Revisar logs de SSH (cualquier login sospechoso)
4. Si hubo acceso indebido: rotar todas las llaves

💡 **Tip pro:** mantenete una nota en el repo o en un documento compartido con la lista de admins activos y sus emails. Cada vez que agregas o remueves alguien, actualiza esa lista.

► DEMO EN VIVO • ~10 MIN

Vamos a darle acceso a un nuevo admin

Generamos un par de llaves SSH desde una máquina nueva, agregamos la pública al servidor, y validamos que la persona puede entrar. Después la quitamos para no dejar accesos abiertos.

- 01 Crear par de llaves: `ssh-keygen -t ed25519 -C "demo@test"`
- 02 Copiar la pública: `cat ~/.ssh/id_ed25519.pub`
- 03 Conectarse al server y editar `/root/.ssh/authorized_keys`
- 04 Validar el login desde la máquina del nuevo admin
- 05 Auditar: `last -F` para ver el acceso recién registrado
- 06 Limpiar: borrar la línea agregada y verificar que ya no entra

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





CONTINUIDAD DEL SERVICIO

Backup & disaster recovery

Si el droplet se pierde mañana, ¿qué necesitas para volver a tener todo funcionando? 5 cosas para backupear, 1 lista de lo que NO hace falta, plan de DR completo y dónde guardar los backups.

5 COSAS A BACKUPEAR

- 1. DB de analytics (diaria)
- 2. Certificados SSL (mensual)
- 3. Configs de servicios (cada cambio)
- 4. Datos GTFS / OSM / mbtiles (al actualizar)
- 5. `authorized_keys` (cada cambio)

LO QUE NO HACE FALTA

- `graph.obj` — se regenera
- `photon_data/` — se descarga
- Imágenes Docker — se rebuildean
- Logs de containers — efímeros
- Cache del proxy — 5s

RESTO DE LA SECCIÓN

- Plan completo de DR (server perdido)
- Dónde guardar los backups
- Estrategia 3-2-1



Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
giz - Gesellschaft für Internationale Zusammenarbeit





BACKUP · 1 / 8 · ALTA · DIARIA

1. Base de datos de analytics

PostgreSQL con la tabla `requests` que YARP loggea. Si se pierde, se pierde todo el histórico de tráfico — no se puede recuperar de otra fuente.

Backup diario obligatorio.

DUMP MANUAL

```
docker exec server-db-1 \  
  pg_dump -U analytics analytics \  
  | gzip > backups/db-$(date +%F).sql.gz
```

CRON DIARIO (3 AM)

```
# /etc/crontab  
0 3 * * * root cd /root && \  
  docker exec server-db-1 \  
    pg_dump -U analytics analytics \  
    | gzip \  
    > /root/backups/db-$(date +%F).sql.gz  
  
# Rotar: borrar > 30 días  
5 3 * * * root \  
  find /root/backups -name 'db-*.sql.gz' \  
    -mtime +30 -delete
```

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale Zusammenarbeit





BACKUP · 2 / 8 · MEDIA · MENSUAL

2. Certificados SSL

LettuceEncrypt los re-emite automáticamente, así que técnicamente no son críticos. Pero un backup evita golpear los **rate limits** de Let's Encrypt si reinstalas todo y necesitas emitir muchos certs juntos.

BACKUP

```
tar czf backups/lettuce-$(date +%F).tar.gz \  
/root/server/data/lettuce-encrypt/
```

RESTORE

```
# Antes de levantar YARP:  
tar xzf backups/lettuce-2026-04-29.tar.gz \  
-C /  
  
# YARP los detecta automáticamente al levantar  
cd /root/server  
docker compose up -d
```

Si el backup tiene certs vigentes (no expirados), YARP los usa hasta su renovación natural — no llama a Let's Encrypt.

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





BACKUP · 3 / 8 · ALTA · CADA CAMBIO

3. Configs de servicios

Los `docker-compose.yml` y `.env` de cada servicio + el `appsettings.json` de YARP. Sin esto, levantar todo de nuevo es un ejercicio de prueba y error.

LO QUE HAY QUE GUARDAR

```
backups/server-config/
├─ appsettings.json  ← YARP
└─ docker-compose.yml ← YARP

# Cada servicio: su .env + docker-compose
/root/otp/.env
/root/otp/docker-compose.yml
/root/photon/.env
/root/photon/docker-compose.yml
... (repetir para cada svc)
```

BACKUP COMPLETO

```
tar czf backups/configs-$(date +%F).tar.gz \
  /root/*/docker-compose.yml \
  /root/*.env \
  /root/server/data/config/

# Estos archivos también deberían estar
# versionados en git si son razonables
# (ojo con secretos en .env)
```

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale Zusammenarbeit





BACKUP · 4 / 8 · ALTA · AL ACTUALIZAR

4. Datos GTFS / OSM / mbtiles

Los inputs canónicos del sistema. Buena noticia: ya están en el repo ([server/data/](#)). Backupear es [git push](#) . Cada vez que se actualizan en el server, sincronizar al repo.

SINCRONIZAR AL REPO

```
SERVER=root@trujillo.trufi.dev

# Bajar todos los inputs canónicos
scp $SERVER:/root/otp/data/trujillo.gtfs.zip \
  server/data/
scp $SERVER:/root/otp/data/trujillo.osm.pbf \
  server/data/
scp $SERVER:/root/tileserv-gl/data/trujillo.mbtiles \
  server/data/
```

COMMIT Y PUSH

```
git add server/data/
git commit -m "Snapshot data $(date +%F)"
git push
```

Total ~26 MB. Considerar Git LFS si crece mucho. Mientras estos archivos vivan en git, el repo es el backup.



Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





BACKUP · 5 / 8 · ALTA · CADA CAMBIO

5. `authorized_keys` + lista de admins

El `/root/.ssh/authorized_keys` tiene las llaves SSH de todos los que pueden entrar. Si alguien renuncia o entra al equipo, hay que actualizarlo.

Backupear cada vez que cambia.

BACKUP

```
# Local
cp /root/.ssh/authorized_keys \
  backups/authorized_keys-$(date +%F)

# A tu máquina
scp root@trujillo.trufi.dev:/root/.ssh/authorized_keys \
  ./backups/authorized_keys-$(date +%F)
```

LISTA DE ADMINS

Mantener una lista **fuera del repo** con quién es quién:

```
# lista-admins.md (en doc compartido)
```

Email	SSH key (comment)	Activo
-----	-----	-----
sam@nexion.com	sam@laptop	sí
carlos@mpt.gob.pe	carlos@cipriano	sí
ana@trufi.org	ana-2023-laptop	NO

Importante: NO commitear esto al repo público. Privacidad + seguridad.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





BACKUP · 6 / 8 · QUÉ EVITAR

Lo que NO hace falta backupear

Errores comunes: backupear archivos enormes que el sistema regenera solo. Ocupas espacio sin valor. Estos 5 archivos/folders son derivados — no source — y volverán a aparecer al levantar los servicios.

ARCHIVOS DERIVADOS

- `graph.obj` de OTP (~20 MB) — se regenera del GTFS + OSM al primer arranque
- `photon_data/` (varios GB) — se descarga desde GraphHopper la primera vez
- Imágenes Docker (~2 GB) — se rebuildean del compose
- Logs de containers — efimeros, no críticos
- Cache del proxy GTFS-RT — se reconstruye en 5s

POR QUÉ IMPORTA

Si por error haces backup de `photon_data/` diariamente:

- Cada backup pesa varios GB
- El backup tarda mucho más
- El destino (S3, etc.) cobra por GB
- El restore es más lento
- Y al final, lo único que quieres es la lista de inputs — no el índice derivado

Regla simple: si se puede regenerar sin pérdida desde otros archivos, no lo backupees.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale Zusammenarbeit





BACKUP · 7 / 8 · DISASTER RECOVERY

Server perdido — el plan de 10 pasos

Si el droplet se pierde por completo (hardware, hack, error humano), este es el playbook para volver a producción. Con todos los backups en mano, ~1 hora si tienes el flujo automatizado, ~3 horas a mano.

PASOS 1-5: NUEVO SERVER

- Crear nuevo droplet (Ubuntu 22.04, ≥4 GB RAM, idealmente igual al anterior)
- Apuntar el A record DNS al nuevo IP
- Instalar Docker + docker-compose
- `git clone --recurse-submodules` el repo en `/root/`
- Mover los folders del repo a `/root/<svc>/` según convención

PASOS 6-10: DATA Y ARRANQUE

- Subir data: `scp server/data/* root@host:/root/<svc>/data/`
- Restaurar DB: `gunzip -c db-backup.sql.gz | psql ...`
- Restaurar certs: `tar xzf lettuce-encrypt.tar.gz -C /`
- Levantar todo: `docker compose up -d` en cada folder
- Re-agregar `authorized_keys` de los admins

Tiempo estimado: ~1h con runbook scripteado, ~3h a mano. La graph.obj de OTP toma 2-3 min adicionales en regenerarse.

Con el apoyo de



Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
GIZ GmbH
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH





BACKUP · 8 / 8 · ANALYTICS DB

Backup de la DB de analytics — dump y restore en local

El backup que más vas a hacer. Dos formas de obtenerlo: bajar un dump ya generado del server, o hacer un `pg_dump` live en el momento. Después lo restaurás en una Postgres local para hacer queries de análisis sin tocar producción.

1 HACER EL DUMP Y BAJARLO

Una sola línea: corre `pg_dump` dentro del container `server-db-1`, lo comprime y lo descarga directo a tu máquina (sin escribir nada en el server).

```
SERVER=root@trujillo.trufi.dev

ssh $SERVER \
  'docker exec server-db-1 \
    pg_dump -U analytics analytics | gzip' \
  > analytics-$(date +%F).sql.gz

# Verificar
ls -lh analytics-*.sql.gz
```

2 RESTAURAR EN POSTGRES LOCAL

```
# Levantar Postgres en local
# (misma imagen que producción)
docker run -d --name analytics-local \
  -e POSTGRES_USER=analytics \
  -e POSTGRES_PASSWORD=analytics \
  -e POSTGRES_DB=analytics \
  -p 5432:5432 \
  postgres:16-alpine

# Restaurar el dump
gunzip -c analytics-2026-04-29.sql.gz \
  | docker exec -i analytics-local \
    psql -U analytics -d analytics

# Conectarte para hacer queries
docker exec -it analytics-local \
  psql -U analytics -d analytics

# o desde tu cliente favorito (DBBeaver,
# TablePlus, etc.) en localhost:5432
# user: analytics · pass: analytics
```

Trabajar sobre la copia local: rápido, sin riesgo de romper producción, y puedes probar queries pesadas sin afectar a YARP.

Con el apoyo de



PERÚ

Ministerio
de Transportes
y ComunicacionesImplementada por
giz
GIZ - Deutsche Gesellschaft
für Internationale ZusammenarbeitEmbajada de Suiza en el Perú
Secretaría de Estado para Asuntos Económicos SECO



Gracias

¿Preguntas?

 github.com/leoguti/GTFS-Trujillo

 [linkedin.com/in/samuel-rioja](https://www.linkedin.com/in/samuel-rioja)

Con el apoyo de



PERÚ

Ministerio
de Transportes
y Comunicaciones



Implementada por
giz
Deutsche Gesellschaft
für Internationale
Zusammenarbeit (GIZ) GmbH

